TÉCNICAS DE PROGRAMACIÓN ORIENTADA A OBJETOS

UNIDAD 1:
Fundamentos de Programación Orientada a Objetos

SESIÓN 5:
Requerimientos de Sistemas, Historias de Usuario y Diagramas UML

Dr. Eric Gustavo Coronel Castillo
eric.coronel@upn.pe

MOTIVACIÓN

DESAFIO DEL DIA

Construir sin entender el problema es programar a ciegas: primero precisión, luego implementación.

MOTIVACIÓN

IDEA FUERZA

Los requerimientos, historias de usuario y UML convierten necesidades en especificaciones; sin eso, el “código” no asegura valor.

LOGRO DE LA UNIDAD 1

Al finalizar la unidad, el estudiante desarrolla casos aplicando correctamente los principios de programación orientada a objetos, demostrando buenas prácticas de programación y utilizando sistemas de control de versiones para gestionar eficazmente su código.

LOGRO DE LA SESIÓN

Al finalizar la sesión, el estudiante será capaz de:

1. Identificar y documentar requerimientos funcionales y no funcionales de un sistema de software.
2. Elaborar Historias de Usuario siguiendo el formato estándar (rol, acción, valor) con criterios de aceptación.
3. Construir Diagramas UML (casos de uso, clases y secuencia) que modelen la solución a un problema real.

REFLEXIÓN

Activando Saberes Previos

Reflexiona y responde antes de comenzar:

1. Cuando un cliente te pide que desarrolles un sistema informático: ¿Cómo sabrías exactamente qué debe hacer ese sistema? ¿Qué información necesitarías recopilar antes de escribir una sola línea de código?

2. ¿Alguna vez has leído un plano de una casa o el manual de instrucciones de un producto? ¿En qué se parecería un diagrama de software a esos documentos? ¿Para qué crees que sirven esos diagramas?

RESPUESTAS

Fundamento Teórico

Respuesta 1: Limitaciones de los arreglos y necesidad de colecciones

Los arreglos en Java tienen tamaño fijo definido en tiempo de compilación. El Java Collections Framework (JCF) resuelve esto proporcionando estructuras de datos dinámicas. Según Bloch (2018), "las colecciones son preferibles a los arreglos porque son más flexibles y expresan mejor la intención del programador". El JCF fue introducido en Java 2 como un conjunto unificado de interfaces y clases para manejar grupos de objetos (Oracle, 2024).

Oracle. (2024). Java Collections Framework Overview. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/doc-files/coll-overview.html

Respuesta 2: List, Set y Map – diferencias fundamentales

Según Vegas Gertrudix (2021), las tres interfaces principales del JCF tienen propósitos distintos: List (fila de pacientes) mantiene orden de inserción y permite duplicados; Set (colores únicos) no admite elementos duplicados y no garantiza orden; Map (precio por código) asocia claves únicas con valores. Esta diferenciación permite al programador seleccionar la estructura óptima según el problema, siguiendo el principio de adecuación semántica.

Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación. eLibro. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222664

Estructura	Duplicados	Orden	Acceso por clave
List	✓ Sí	Inserción	No
Set	✗ No	No garantizado	No
Map	Claves únicas	Depende impl.	✓ Sí

01 ¿Cómo saber qué debe hacer el sistema?

Los requerimientos de software son descripciones de lo que el sistema debe hacer: los servicios que ofrece y las restricciones de su operación. Recopilarlos es la actividad central de la Ingeniería de Requerimientos.

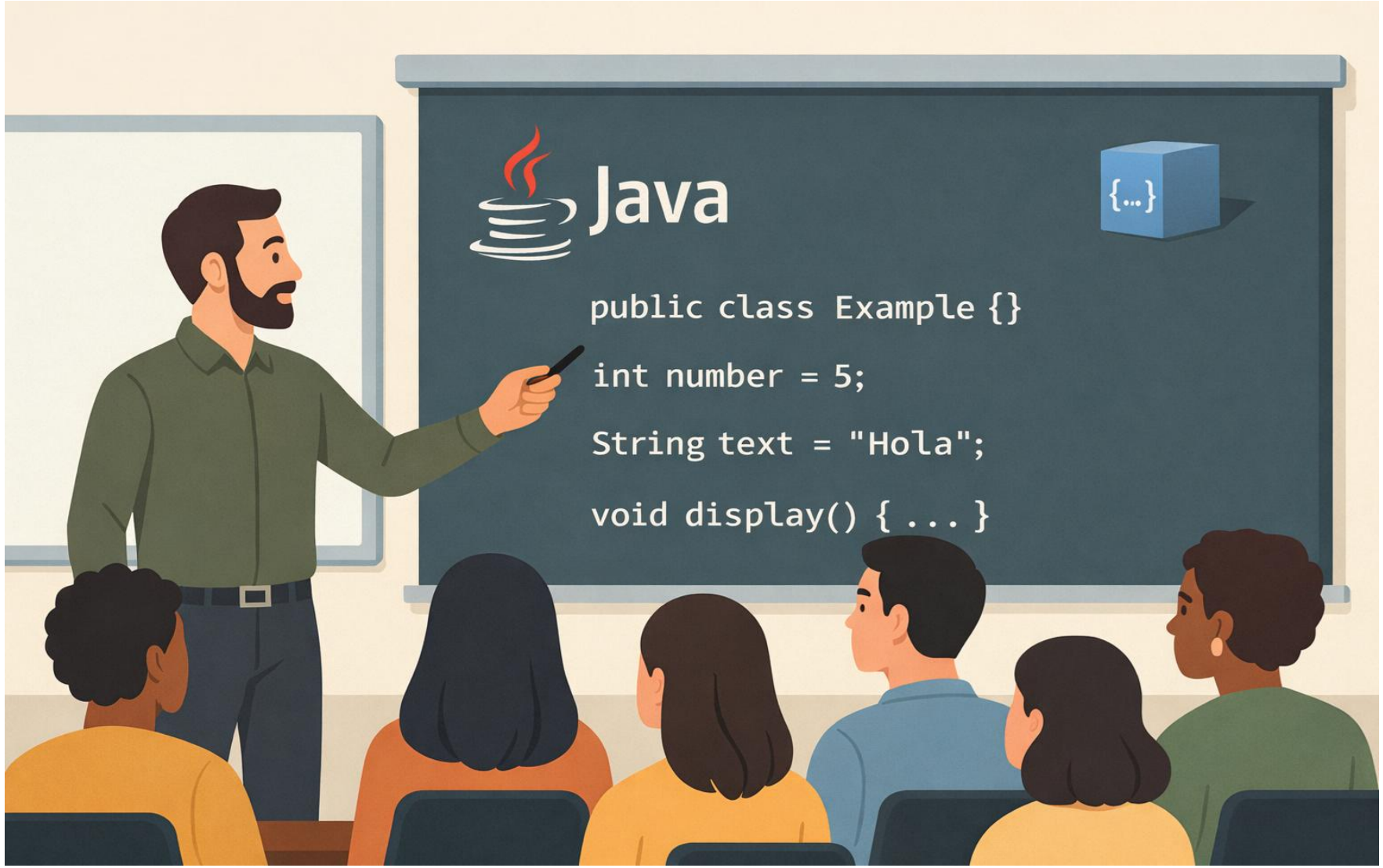
Sommerville, I. (2016). Software Engineering (10.ª ed., p. 83). Pearson. https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003364

02 ¿Para qué sirven los diagramas en software?

UML (Unified Modeling Language) es un lenguaje de modelado estandarizado (ISO/IEC 19501) que permite visualizar, especificar, construir y documentar los artefactos de un sistema de software, de la misma manera que los planos sirven para construir edificios.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML) v2.5.1. https://www.omg.org/spec/UML/2.5.1

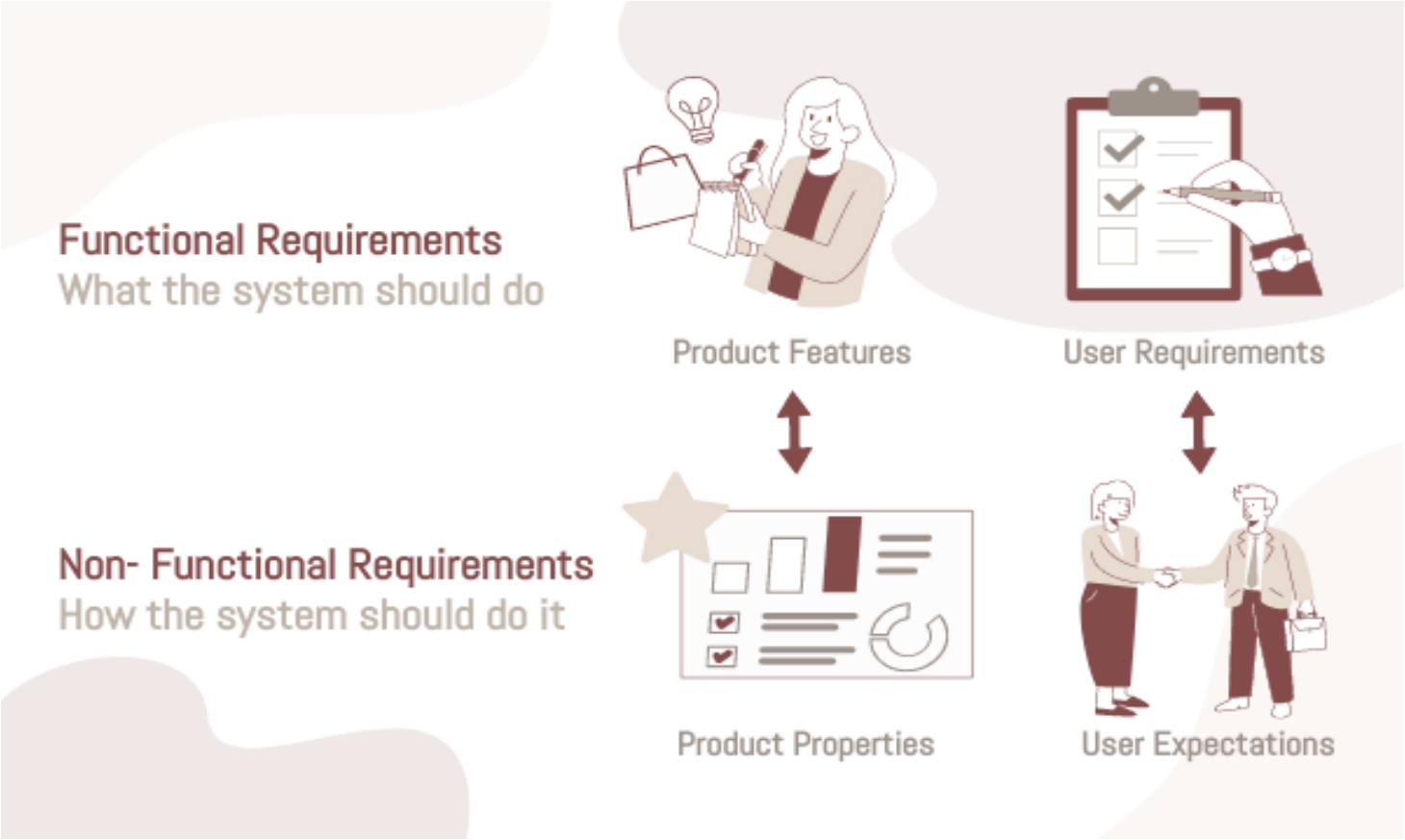
DESARROLLO


Java
{...}
public class Example {}
int number = 5;
String text = "Hola";
void display() { ... }

Requerimientos de Sistemas

Dos perspectivas, un solo sistema

Los requerimientos funcionales definen las características del producto y lo que el usuario espera que ocurra. Los no funcionales establecen las propiedades del producto y las expectativas de calidad que lo hacen confiable.



Requerimientos de Sistemas

¿Qué es un Requerimiento?

Una declaración de un servicio o restricción que el sistema debe satisfacer.

Puede ser de alto nivel (abstracta) o detallada y matemática.

Ambas visiones son requerimientos legítimos según el contexto del proyecto.

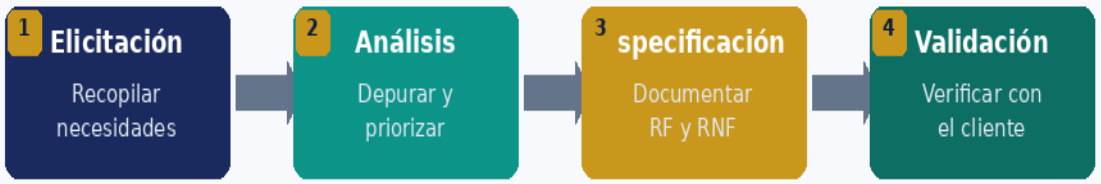
Sommerville (2016, p. 83).

Tipos de Requerimientos

RF – Funcionales: Lo que el sistema HACE

RNF – No Funcionales: Restricciones de calidad

Proceso de Ingeniería de Requerimientos



Adaptado de: Sommerville, I. (2016). Software Engineering (10.ª ed.). Pearson.

Proceso de Ingeniería de Requerimientos.
Adaptado de Sommerville (2016).

Tipos de Requerimientos

Requerimientos Funcionales (RF)	Requerimientos No Funcionales (RNF)
RF01: El sistema permitirá registrar nuevos usuarios con nombre, correo y contraseña.	RNF01 (Rendimiento): El sistema responderá a las consultas en menos de 2 segundos.
RF02: El sistema mostrará un listado de todos los productos disponibles.	RNF02 (Seguridad): Las contraseñas serán cifradas con algoritmo bcrypt.
RF03: El sistema generará un reporte de ventas en formato PDF.	RNF03 (Usabilidad): La interfaz será accesible en navegadores Chrome, Firefox y Edge.
RF04: El sistema enviará un correo de confirmación al realizar una compra.	RNF04 (Disponibilidad): El sistema estará disponible el 99.9% del tiempo.

Fuente: Sommerville, I. (2016). Software Engineering (10.ª ed., pp. 83-100). Pearson.

Historias de Usuario

Video sobre historias de Usuario

Objetivos:

- ¿Qué son las historias de Usuario?
- ¿Cómo elaborar las Historias?
- Criterios de Calidad de las Historias
- Ventajas y Desventajas

Historias de Usuario

¿Qué es una Historia de Usuario?

Una historia de usuario es una descripción breve y simple de una funcionalidad contada desde la perspectiva del usuario final o cliente (Cohn, 2004).

Cohn, M. (2004). User Stories Applied. Addison-Wesley.

Formato Estándar (Conextra)

"Como [ROL] quiero [ACCIÓN] para [VALOR/BENEFICIO]"

Ejemplo de Historia de Usuario

HU-001: Registro de Cliente

Como: cliente del sistema

Quiero: registrarme con mis datos personales

Para: acceder a las funcionalidades del sistema

Criterios de Aceptación

✓ El sistema debe validar formato de correo.

✓ La contraseña debe tener mínimo 8 caracteres.

✓ Se muestra mensaje de confirmación al registrarse.

Schwaber & Sutherland (2020). The Scrum Guide. https://scrumguides.org

Introducción a UML

¿Qué es UML?

UML (Unified Modeling Language) es un lenguaje estándar (ISO/IEC 19501 / OMG UML 2.5.1) para visualizar, especificar, construir y documentar los artefactos de un sistema de software.

OMG (2017). OMG UML v2.5.1. https://www.omg.org/spec/UML/2.5.1

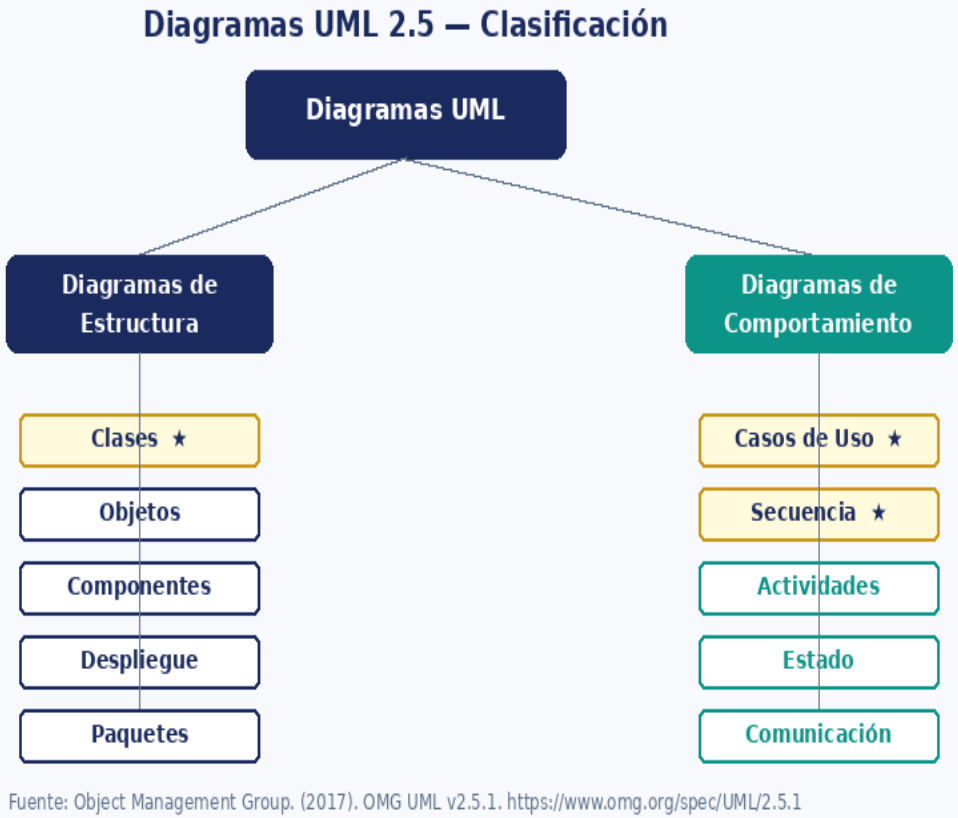
Diagramas clave en el curso:

Casos de Uso – ¿Qué hace el sistema?

Clases – ¿Cómo se estructuran?

Secuencia – ¿Cómo interactúan?

Clasificación de Diagramas UML 2.5



Clasificación UML 2.5. Fuente: OMG (2017).

Diagrama de Casos de Uso

Diagrama de Casos de Uso

Representa las interacciones entre actores (personas/sistemas externos) y las funcionalidades del sistema (Booch et al., 2005).

Elementos principales:

Actor: Rol externo al sistema (persona u otro sistema).

Caso de Uso: Funcionalidad prevista (representado en elipse).

Asociación: Relación entre actor y caso de uso (línea).

«include»: Un CU incluye obligatoriamente otro.

«extend»: Un CU extiende opcionalmente otro.

Booch et al. (2005). The UML User Guide. Addison-Wesley.

Ejemplo: Sistema Clínica Veterinaria

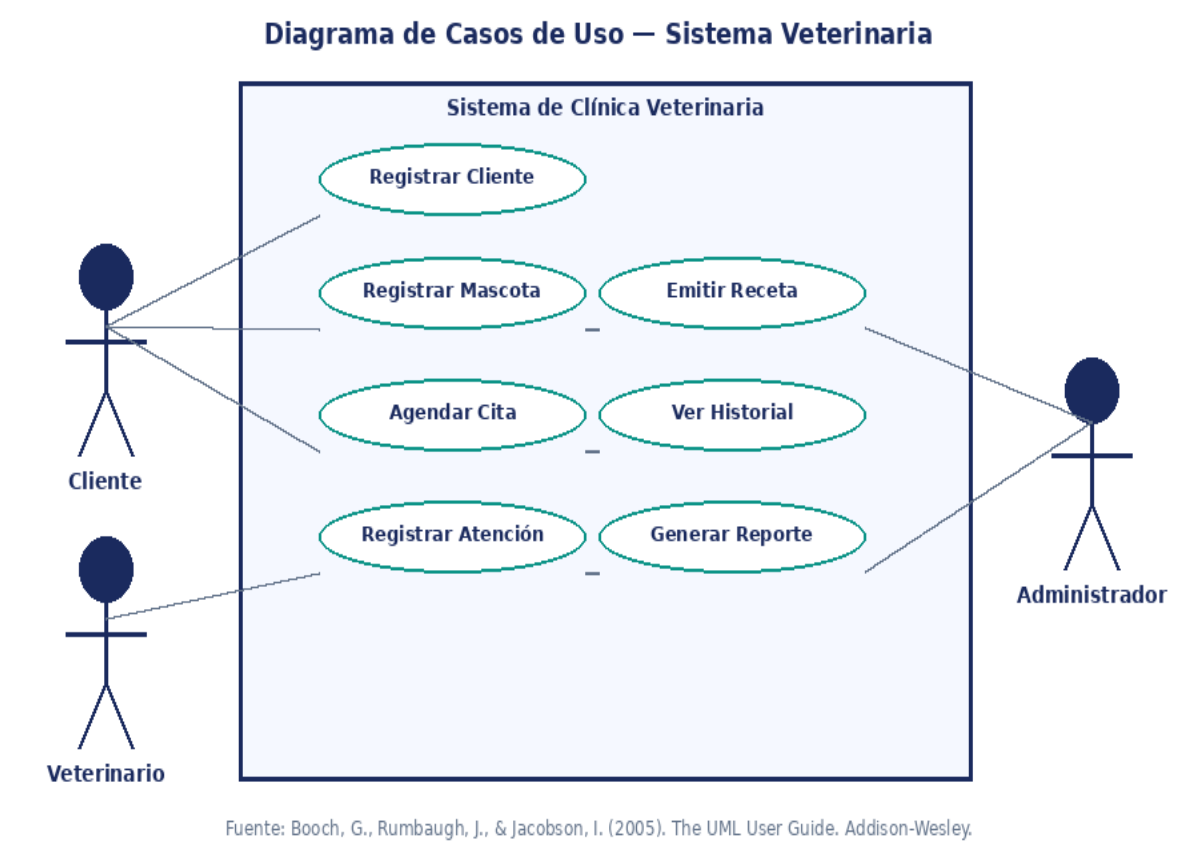


Diagrama de Casos de Uso. Elaboración propia basada en Booch et al. (2005).

Diagrama de Clases

Diagrama de Clases

El diagrama más utilizado en OOP. Modela la estructura estática del sistema: clases, atributos, métodos y relaciones (OMG, 2017).

Elementos clave:

Clase: Rectángulo: nombre / atributos / métodos.

+ público: Accesible desde cualquier lugar.

- privado: Solo accesible dentro de la clase.

 # protegido: Accesible en la clase y subclases.

Herencia: Flecha triángulo vacío: subclase → superclase.

Multiplicidad: 1, * (muchos), 0..1 (opcional).

OMG (2017). OMG UML v2.5.1. https://www.omg.org/spec/UML/2.5.1

Ejemplo: Sistema Clínica Veterinaria

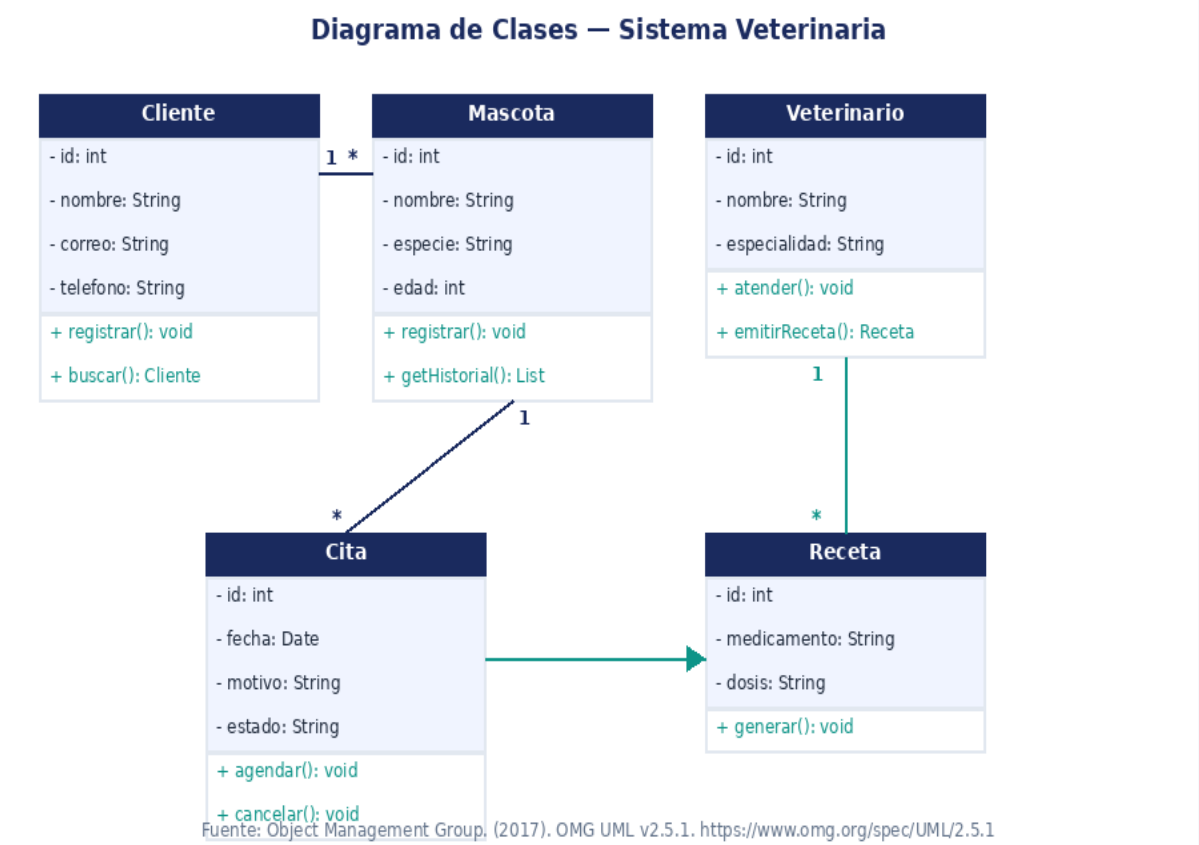


Diagrama de Clases. Elaboración propia basada en OMG UML 2.5.1 (2017).

Arquitectura en Capas: Concepto

Organización del Diagrama de Clases por responsabilidad

¿Por qué organizar las clases en capas?

Un Diagrama de Clases organizado en capas no solo modela la estructura, sino que también comunica la arquitectura del sistema y las responsabilidades de cada componente.

Las 3 capas del curso:

VIEW: Presentación. Formularios, ventanas. No contiene lógica.

CONTROLLER: Control. Coordina el flujo entre la Vista y el Servicio.

SERVICE: Lógica de negocio. Contiene las reglas del dominio.

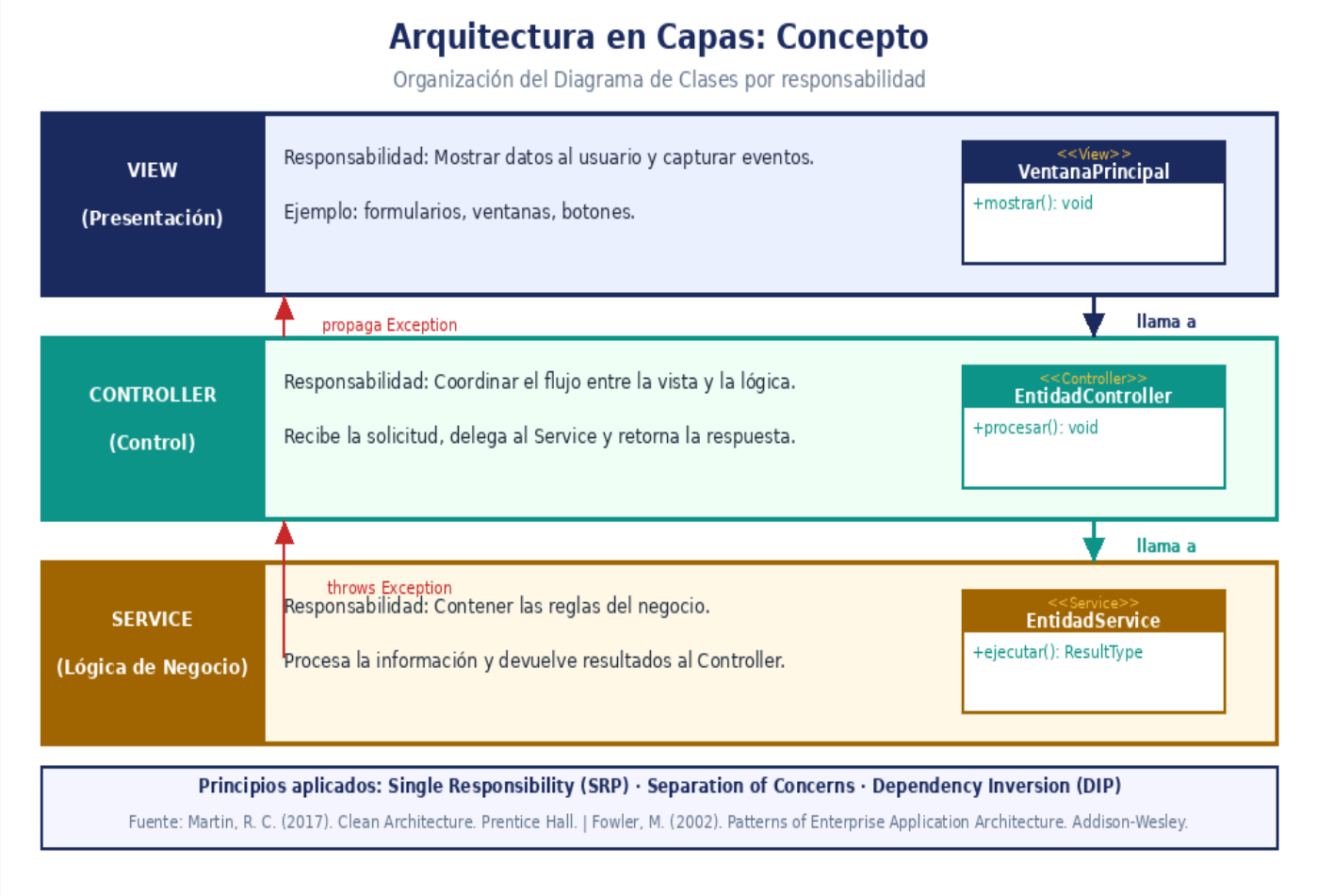


Figura 7. Arquitectura View-Controller-Service. Elaboración propia basada en Martin (2017) y Fowler (2002).

Principios SOLID: SRP · Separation of Concerns · DIP

Martin (2017). Fowler (2002).

Arquitectura en Capas: Ejemplo

Sistema Calculadora — Diagrama de Clases en 3 capas sin base de datos

Lectura del Diagrama

VentanaCalculadora (VIEW)

Solo muestra campos y botones. Llama al Controller al hacer clic.

CalculadoraController

Recibe los datos de la View, delega al Service y retorna el resultado.

CalculadoraService

Contiene TODA la lógica. dividir() lanza ArithmeticException si b=0.

Propagación de errores

Exception: Service → Controller → View sin necesidad de base de datos.

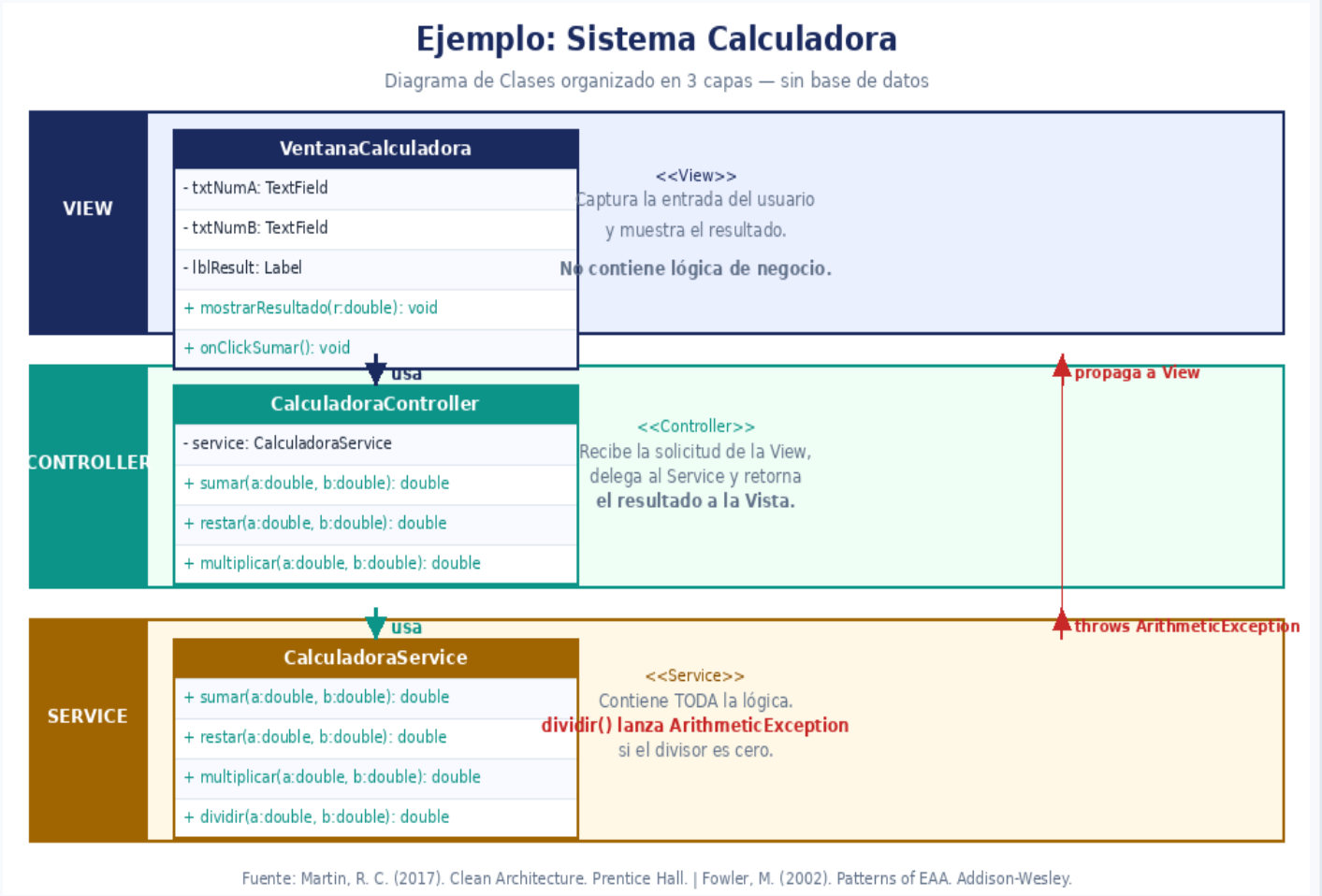
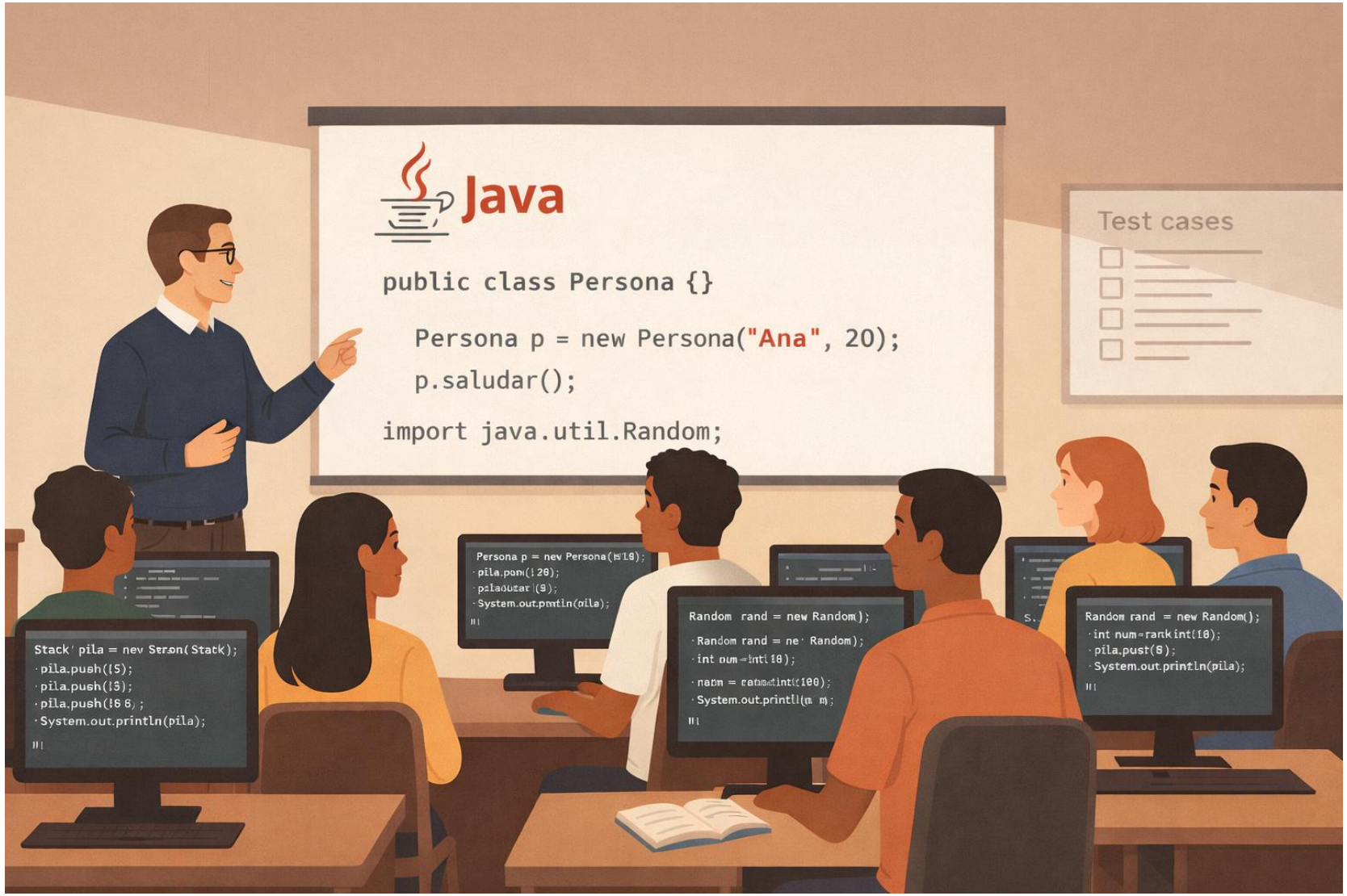


Figura 8. Diagrama de Clases – Calculadora en capas. Elaboración propia basada en Martin (2017).

Martin, R. C. (2017). Clean Architecture. Prentice Hall.
Fowler, M. (2002). Patterns of EAA. Addison-Wesley.

ACTIVIDAD EN CLASE



Ejercicio 1: Sistema de Gestión de Tareas Personales

El TareaService gestiona una lista de tareas precargada en un ArrayList<Tarea>. No hay base de datos: los datos viven en memoria durante la ejecución. Implementa la arquitectura View → Controller → Service.

Clases a implementar

Tarea (Model):
- id: int
- titulo: String
- completada: boolean

TareaService:
- tareas: ArrayList<Tarea>
+ listar(): List<Tarea>
+ agregar(t: Tarea): void
+ completar(id: int): void
+ eliminar(id: int): void

TareaController:
+ listarTareas(): List<Tarea>
+ agregarTarea(titulo: String): void
+ completarTarea(id: int): void

VentanaTareas (View):
+ mostrarLista(): void
+ onClickAgregar(): void

Tareas a desarrollar

1. **Precargar datos:** En TareaService, inicializa el ArrayList con al menos 5 tareas en el constructor.
2. **Listar tareas:** La View debe mostrar todas las tareas con su estado (pendiente / completada).
3. **Agregar tarea:** El usuario ingresa un título y el sistema agrega la tarea al ArrayList.
4. **Completar tarea:** Busca la tarea por id y cambia completada = true.
5. 🏆 **Reto extra:** Agrega el método filtrarPendientes() que retorne solo las tareas no completadas.

Diseña primero el Diagrama de Clases en papel, luego implementa en Java.

Ejercicio 2: Sistema Conversor de Unidades

Implementa un Conversor de Unidades aplicando la arquitectura View → Controller → Service. Toda la lógica matemática debe estar en ConversorService. La View solo captura el valor y la unidad de origen.

Estructura de Clases

ConversorService:
+ kmAMillas(km: double): double
+ celsiusAFahrenheit(c: double): double
+ kgALibras(kg: double): double
+ litrosAGalones(l: double): double

ConversorController:
- service: ConversorService
+ convertir(valor: double, tipo: String): double

VentanaConversor:
- txtValor: TextField
- cmbTipo: ComboBox
+ onClickConvertir(): void
+ mostrarResultado(r: double): void

Fórmulas a implementar

Km → Millas

```
millas = km × 0.621371
```

°C → °F

```
fahrenheit = (c × 9/5) + 32
```

Kg → Libras

```
libras = kg × 2.20462
```

Litros → Galones

```
galones = litros × 0.264172
```

🏆 **Reto:** Valida que el valor ingresado sea mayor que cero. Si no, lanza IllegalArgumentException en el Service y muéstralo en la View.

Implementa primero kmAMillas() y pruébala, luego agrega las demás.

CONCLUSIONES

Lo que aprendimos

1

Los requerimientos son la base del software

Un sistema de calidad comienza por la correcta elicitación, análisis y especificación de requerimientos (funcionales y no funcionales). Sin esta base, el diseño y la implementación carecen de dirección.

Sommerville (2016, p. 83).

2

Las Historias de Usuario centran el desarrollo en el valor real

El formato «Como… quiero… para…» obliga al equipo a pensar en el beneficio que aporta cada funcionalidad, alineando el producto con las necesidades reales del usuario antes de codificar.

Cohn (2004). Schwaber & Sutherland (2020).

3

UML y la arquitectura en capas comunican el diseño con precisión

Los diagramas de clases organizados en capas (View–Controller–Service) no solo modelan la estructura, sino que comunican la separación de responsabilidades (SRP) y facilitan el mantenimiento del sistema.

OMG (2017). Martin (2017). Fowler (2002).

4

Las colecciones permiten persistencia en memoria sin base de datos

Durante las primeras semanas, un ArrayList precargado en el Service es suficiente para implementar y probar la lógica de negocio completa, siguiendo la misma arquitectura que se usará con base de datos.

Blanco (2020). Vegas Gertrudix (2021).

Conclusiones de la Sesión

1

Los requerimientos son la base del software

Un sistema de calidad comienza por la correcta elicitación, análisis y especificación de requerimientos (funcionales y no funcionales). Sin esta base, el diseño y la implementación carecen de dirección.

Sommerville (2016, p. 83).

2

Las Historias de Usuario centran el desarrollo en el valor real

El formato «Como… quiero… para…» obliga al equipo a pensar en el beneficio que aporta cada funcionalidad, alineando el producto con las necesidades reales del usuario antes de codificar.

Cohn (2004). Schwaber & Sutherland (2020).

3

UML y la arquitectura en capas comunican el diseño con precisión

Los diagramas de clases organizados en capas (View–Controller–Service) no solo modelan la estructura, sino que comunican la separación de responsabilidades (SRP) y facilitan el mantenimiento del sistema.

OMG (2017). Martin (2017). Fowler (2002).

4

Las colecciones permiten persistencia en memoria sin base de datos

Durante las primeras semanas, un ArrayList precargado en el Service es suficiente para implementar y probar la lógica de negocio completa, siguiendo la misma arquitectura que se usará con base de datos.

Blanco (2020). Vegas Gertrudix (2021).

¿PREGUNTAS O COMENTARIOS?

¿PREGUNTAS?

¡El momento de despejar dudas es ahora!

Tu pregunta no solo aclara tu duda,
también enriquece el aprendizaje de todos

COMPROBACIÓN DEL LOGRO

COMPROBACIÓN DEL LOGRO

Cuestionario en la plataforma

Nivel	Rango
Nivel 4	17 – 20
Nivel 3	13 – 16
Nivel 2	9 – 12
Nivel 1	0 - 8

COMPROBACIÓN DEL LOGRO

Criterio	Nivel 4 (Excelente)	Nivel 3 (Bueno)	Nivel 2 (Regular)	Nivel 1 (Deficiente)
Comprensión conceptual	Demuestra comprensión completa de los conceptos clave.	Comprende la mayoría de los conceptos clave.	Muestra comprensión parcial, con algunas confusiones.	No demuestra comprensión clara de los conceptos.
Aplicación de conocimientos	Aplica correctamente los conceptos en ejemplos o escenarios.	Aplica conceptos con algunos errores menores.	Aplica de forma superficial o con errores evidentes.	No logra aplicar los conceptos o lo hace incorrectamente.
Claridad y precisión en respuestas	Las respuestas son claras, bien estructuradas y precisas.	Las respuestas son mayormente claras, con leves imprecisiones.	Respuestas poco claras o con ideas mal organizadas.	Respuestas confusas, incompletas o incoherentes.
Cobertura del contenido esperado	Responde completamente las 5 preguntas, desarrollando bien cada una.	Responde correctamente 4 de las 5 preguntas.	Responde correctamente 2 o 3 preguntas.	Responde solo 1 pregunta correctamente o no responde.

BIBLIOGRAFIA REFERENCIAL

REFERENCIAS BIBLIOGRÁFICAS

Sommerville, I. (2016). Software Engineering (10.ª ed.). Pearson.
https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003364

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML) v2.5.1.
https://www.omg.org/spec/UML/2.5.1

Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley.

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Scrum.org.
https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2.ª ed.). Addison-Wesley.

Vegas Gertrudix, J. M. (2021). Java 17: fundamentos prácticos de programación. Alfaomega.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222664

Jiménez de Parga, C. (2021). UML: arquitectura de aplicaciones en Java, C++ y Python.
https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/222720

GRACIAS

UPN

UNIVERSIDAD
PRIVADA
DEL NORTE